Practice Test 4

Solutions and marking guide

20 marks + 2 bonus. Award partial credit for valid intermediate reasoning and follow-through from an earlier error. Accept equivalent wording, transpose conventions and sign conventions that are applied consistently. Figures are numpy LIF *rate-model* illustrations, not Nengo runs; neural traces are not unique, and only the labelled features are marked.

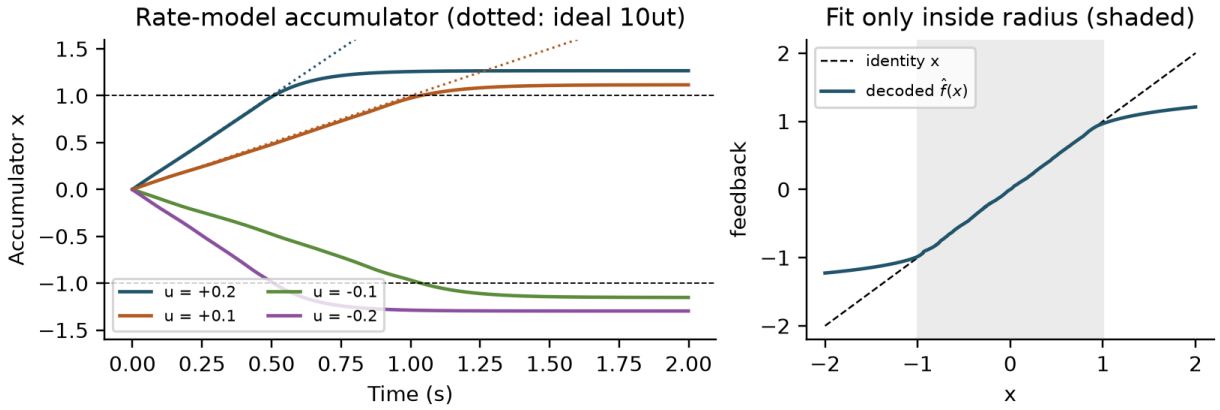
Question 1 [4] – accumulator

(a) With matched synapses the NEF mapping is $A' = \tau A + I$, $B' = \tau B$. Here $A' = 1$ and $B' = 1$, so

$$A = 0, \qquad B = 1/\tau = 10 \text{ s}^{-1}, \qquad x(t) = 10\,u\,t.$$

0.5 for A and B from the mapping, **0.5** for $x(t)$. The network integrates the evidence. Because the input was not pre-scaled by τ, it is integrated ten times faster than $\dot{x} = u$.

(b) Ideal ramps have slopes ± 2 and ± 1 per second. They reach ± 1 at $t = 1/(10|u|)$: 0.5 s for $|u| = 0.2$ and 1 s for $|u| = 0.1$. The simulated traces follow the ramps, then level off slightly beyond ± 1. Stronger evidence decides earlier and sits slightly further out. **Sketch [1]:** four ordered ramps with the correct crossing times [0.5]; plateaus near ± 1 [0.5]. **Mechanism [1]:** The recurrent decoders were optimized only over the radius $[-1, 1]$. Beyond it, no further neurons are recruited and LIF rates flatten, so the decoded feedback satisfies $\hat{f}(x) < x$ for $x > 1$ [0.5]. The state obeys $\tau\dot{x} = \hat{f}(x) - x + u$, so it stops where the feedback shortfall cancels the input: $x - \hat{f}(x) = u$ [0.5]. "The neurons hit their maximum firing rate" earns only 0.5 without the decoded-feedback argument; the rates keep changing slowly, and the decoder fit fails first.



(c) A neuron fires only when its encoded input exceeds its intercept. Before the decision, the decoded drive into the output population is near 0, so $|y| < 0.4$ and every neuron is below threshold [0.5]. All activities are therefore zero, and a linear decode of zero activity is exactly $\sum_i d_i \cdot 0 = 0$ [0.5]. With intercepts spread over $[-1, 1]$, many neurons fire at $y \approx 0$. Their decoded contributions only approximately cancel, and spiking adds fluctuation. Small decoding ripples from the choice function fall inside the silent dead zone and are suppressed. The cost is that intermediate values of $|y|$ cannot be represented. This is acceptable for an output that only needs 0 or ± 1.

Question 2 [4] – LMU

(a) For $q = 1$, $\theta A = (1)(-1)^1 = -1$ and $\theta B = 1$, so

$$\dot{x} = \frac{u - x}{\theta},$$

a first-order low-pass filter with time constant θ [0.5]. $x(t)$ is an exponentially weighted average of roughly the last θ seconds of input. It is the one-term (constant) Legendre approximation of the window and equals the [0/1] Padé approximation of the delay $e^{-s\theta}$, namely $1/(1 + \theta s)$ [0.5]. Then

$$A' = \tau A + 1 = 1 - \tau/\theta = 0.8, \qquad B' = \tau B = \tau/\theta = 0.2 \quad [0.5].$$

(b) Every instantaneous value in $[-1, 1]$ occurs in both sines, so any static function of $u(t)$ gives the same output in both cases [0.5]. Frequency is a property of the recent history. The LMU state is q Legendre coefficients of the last 0.5 s. That window contains half a cycle of the 1 Hz input but a full cycle of the 2 Hz input, so the windows and their coefficient vectors differ. For example, the window-average coefficient is large for 1 Hz and about 0 for 2 Hz. A static decoder of the state can therefore separate the classes [0.5]. Award 0.5 for "the LMU remembers the past" if the answer does not explain why that information separates the classes.

(c) Sketch [0.5]: high error at $q = 1$–2, a minimum at moderate q (roughly 4–8), then rising error at large q. Any U-shape with justified ends earns full credit; the exact minimum is not determined. **Small q [0.5]:** too few coefficients describe the window's shape. $q = 1$ is only a low-pass filter, whose 1-D oscillations for the two frequencies cover overlapping ranges, so the classes are not separable. **Large q [0.5]:** 1000 neurons must represent a q-dimensional state, so each dimension is represented less accurately. The high-order coefficients have large gains $(2i + 1)/\theta$ and fast dynamics that $\tau = 0.1$ s synapses and the radius cannot implement accurately. Recurrent errors then accumulate and corrupt the useful low-order dimensions.

Question 3 [4] – PES

(a) $\dfrac{\partial E}{\partial d_j} = (\hat{y} - y^d)\, a_j = \varepsilon\, a_j$, so

$$\Delta d_j = -\eta\, \varepsilon\, a_j \qquad [1].$$

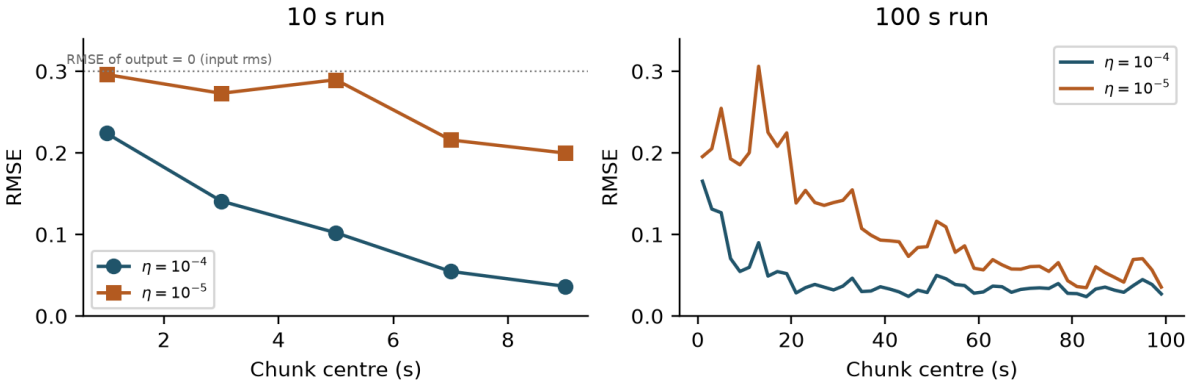
Substituting into $w_{ij} = \alpha_i e_i d_j$ gives

$$\Delta w_{ij} = \alpha_i e_i\, \Delta d_j = -\eta\, (\alpha_i e_i \varepsilon)\, a_j \qquad [0.5],$$

or $-\eta \alpha_i \langle \mathbf{e}_i, \boldsymbol{\varepsilon} \rangle a_j$ in vector form. The term $\alpha_i e_i \varepsilon$ is the error *encoded* into post-neuron i, exactly as if it were an input current. It acts as a modulatory input, comparable to dopaminergic plasticity signals, and is multiplied by the presynaptic activity a_j. The rule is local [0.5].

(b) The weights then step *up* the gradient, so each update increases the error [0.5]. When the output is too large, it is pushed larger, giving positive feedback rather than error correction. The output runs away from the target, with growing magnitude and often the opposite trend, until saturation limits it. The error grows instead of shrinking [0.5].

(c) Sketch [0.5]: $\eta = 10^{-4}$ starts near the input RMS (0.2–0.3), because the output initially computes 0, and falls steadily toward a low floor. $\eta = 10^{-5}$ starts at the same level and decreases about ten times more slowly, staying high during the 10 s. **Floor [0.5]:** any one of the following earns the mark: finite neurons and static distortion; noisy pre and error representations; ongoing weight jitter because $\eta > 0$ keeps reacting to instantaneous noisy errors; or filter lag between the target and the output.



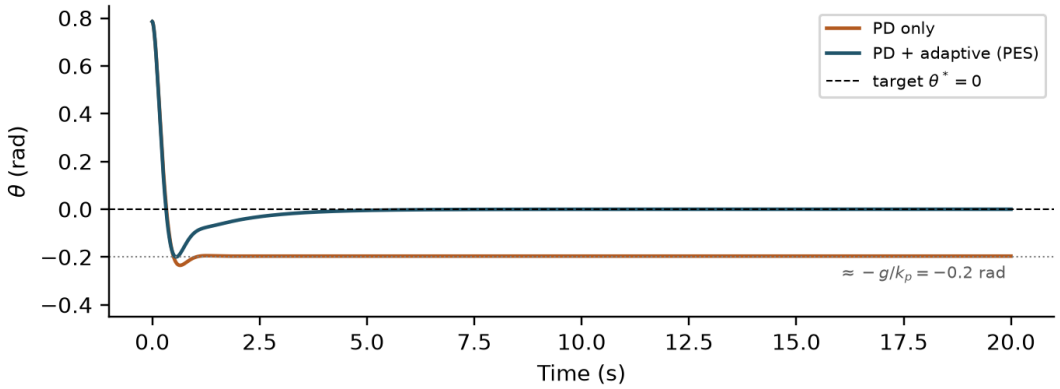
Not asked: over 100 s, the smaller rate reaches a similar or quieter floor, but only after much longer training.

Question 4 [4] – adaptive control

(a) At equilibrium $\dot{\theta} = \ddot{\theta} = 0$, so $k_p(0 - \theta) = mgL\cos\theta$ [0.5]. Hence $\theta = -(g/k_p)\cos\theta \approx -9.8/49 = -0.2$ rad. With $\cos 0.2 \approx 0.98$, the more precise value is about -0.196 rad. The pendulum droops *below* the horizontal, in the direction gravity pulls [0.5].

(b) It learns the unmodelled gravity torque $u_{ad}(\theta, \dot{\theta}) \approx mgL\cos\theta$, a function of the represented state [0.5]. The PD output is the extra torque the system still needs. While compensation is incomplete, the pendulum sits off target and the PD output is nonzero. When compensation is correct, the equilibrium moves to θ^* and the PD output falls to zero, so learning stops at the right solution [0.5]. Because $\Delta d = -\eta\varepsilon a$ lowers the output when $\varepsilon > 0$, use $\varepsilon = -\tau_{PD}$. A positive PD push should increase the adaptive torque [0.5]. This is the same rule as in Question 3, but the error comes from the controller rather than from a known target function.

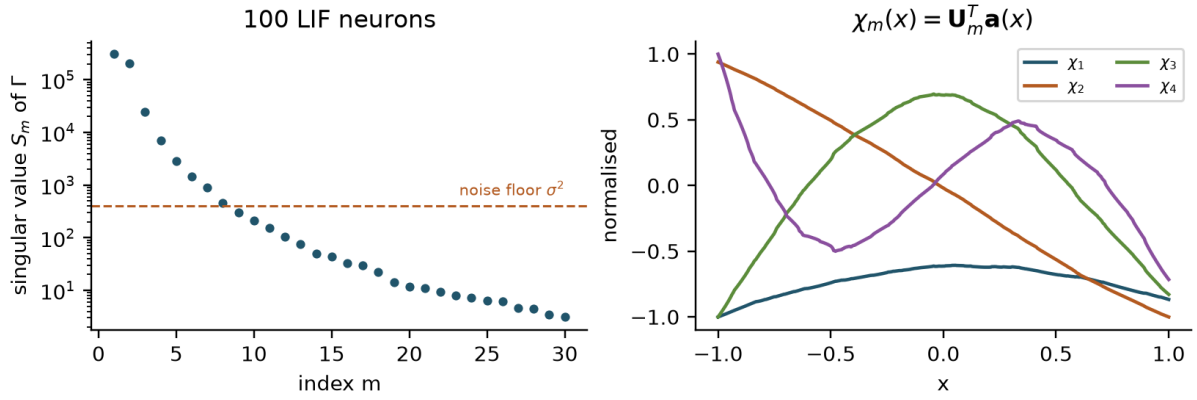
(c) Both traces start at $\pi/4 \approx 0.79$ and fall within about 0.5 s, with a small undershoot [0.5]. PD alone settles and stays at about -0.2 rad [0.5]. With adaptation, the offset decays toward $\theta^* = 0$ over several seconds, at a rate set by η, and remains there [0.5].



Question 5 [4] – SVD

(a) Decompose the Gram matrix of tuning curves, $\Gamma = A^T A$. With the lecture's convention, A is samples $\times$ neurons; with neurons $\times$ samples it is AA^T. Γ is $N \times N$ and measures neuron-neuron correlation [0.5]. Decoders for *any* target solve $\Gamma D = \Upsilon$. Only Υ depends on the function, while Γ depends only on the population. Its decomposition $\Gamma = USU^T$ therefore describes every possible decoding [0.5]. The vectors U_m give orthogonal directions in neuron space, ordered by variance. $\chi_m = AU_m$ are the population's basis functions of x, and S_m gives each one's importance. An SVD of A itself is equivalent: its left singular vectors are U and its singular values are proportional to $\sqrt{S_m}$.

(b) The spectrum decays rapidly, roughly exponentially, over the first several indices, followed by a long, flatter tail [0.5]. A horizontal line marks σ^2. Components above it are usable, and components below it are noise-dominated. In the illustration about 8 of 100 are usable [0.5]. The basis functions $\chi_1, \chi_2, \chi_3, \ldots$ resemble constant, linear and quadratic Legendre-like polynomials (not asked).



(c) Let $r = x - \sum_{n \neq m} \chi_n \Phi_n$. Orthogonality gives $\langle \chi_m r \rangle = c_m$. Adding mode m changes the distortion by

$$\langle (r - \chi_m \Phi_m)^2 \rangle - \langle r^2 \rangle = -2\Phi_m c_m + \Phi_m^2 S_m \qquad [0.5].$$

With $\Phi_m = c_m / S_m$, the distortion falls by c_m^2 / S_m, and the noise term rises by $\sigma^2 \Phi_m^2 = \sigma^2 c_m^2 / S_m^2$ [0.5]. Therefore

$$\Delta E_m = -\frac{c_m^2}{S_m} + \frac{\sigma^2 c_m^2}{S_m^2} = \frac{c_m^2}{S_m}\left(\frac{\sigma^2}{S_m} - 1\right) \qquad [0.5].$$

The mode helps only if $S_m > \sigma^2$. If $S_m \leq \sigma^2$, its normalization $1/S_m$ amplifies noise, and including it *adds* error. The noise therefore sets the effective dimensionality of the representation. Functions that depend on modes below the floor, such as sharp or high-order functions, cannot be decoded reliably. Regularization replaces S_m by $S_m + \sigma^2$ and automatically down-weights these modes [0.5].

Bonus [2]

Write the decoders in the eigenbasis of Γ, $\Phi = U^T \mathbf{d}$. The averaged update $\langle \Delta \mathbf{d} \rangle = -\eta(\Gamma \mathbf{d} - \Upsilon)$ decouples into

$$\dot{\Phi}_m = -\eta \left(S_m \Phi_m - \langle \chi_m y^d \rangle \right).$$

Each mode therefore relaxes exponentially to its optimum with time constant $1/(\eta S_m)$ [1]. The target x is carried mainly by the linear mode χ_2, which has a large S_m, so it is learned quickly. The target x^2 needs the even, quadratic-like mode χ_3, which has a singular value about an order of magnitude smaller, and possibly higher modes. Those components converge correspondingly more slowly [1]. Half credit is available for "x^2 is a more complex function" only if the answer links complexity to smaller singular values.

Timing note

A complete answer needs about 20–27 minutes of writing, plus 2–3 minutes to read and check. Questions 2(c) and 5(c) are the most likely to overrun. Do them last if you are slow.